

Formalized Class Group Computations and Integral Points on Mordell Elliptic Curves

Anne Baanen*

Alex J. Best

Nirvana Coppola

Sander R. Dahmen

t.baanen@vu.nl

alex.j.best@gmail.com

nirvanac93@gmail.com

s.r.dahmen@vu.nl

Vrije Universiteit Amsterdam

Amsterdam, The Netherlands

Abstract

丢番图方程是数论中一个热门且活跃的研究领域。在本文中，我们考虑形式为 $y^2 = x^3 + d$ 的莫德爾（Mordell）方程，其中 d 是一个（给定的）非零整数，并且需要确定所有的整数解 x 和 y 。解决这个问题的一种非初等方法是通过下降法和类群来进行求解。沿着这些思路，我们在 Lean 3 中形式化了几个 $d < 0$ 实例的莫德爾方程的求解过程。为了实现这一目标，我们需要形式化数论中的其他几个本身也很有趣的理论，例如理想范数、二次域和二次环，以及类数的显式计算。此外，我们引入了新的计算策略，以便在二次环及更一般的结构中高效地进行计算。

Keywords: formalized mathematics, algebraic number theory, Diophantine equations, tactics, Lean, Mathlib

1 Introduction

丢番图方程的研究构成了现代代数数论和算术几何的很大一部分 [6]。与数学的相关领域相比，交互式定理证明器中涉及这一主题的内容相对较少。部分原因可能是缺乏除了初等数论之外的工作所需的已形式化的先决条件，例如代数数域理论，包括类群和单位群。此外，许多丢番图方程需要显式计算才能有效求解。如今，这些计算通常由尚未被形式化验证的计算机代数系统执行，如 Mathematica、Magma、Pari/GP 和 SageMath。

在本文中，我们研究被称为莫德爾（Mordell）方程的一类丢番图方程：

$$y^2 = x^3 + d, \quad x, y \in \mathbb{Z}, \quad (1)$$

并对各种非零参数 $d \in \mathbb{Z}$ 的实例形式化了其解的完整描述（第 5 节）。我们在一些基础代数数论之上（第 2 节），使用二次数域（第 3 节）和类群（第 4 节）的显式计算对解进行分类。以前从未在证明助手中进行过此类计算。这些论证的复杂性要求仔细建立理论，并开发抽象结构以在没有极端开销的情况下证明这些结果。

我们报告了形式化是如何进行的，并强调了我们工作的三个重要成果：首先是形式化了一些不平凡丢番图方

程的显式解。其次，我们开发了几个本身也有独立意义的其他理论，例如二次域理论、理想范数和类群计算。第三，我们还建立了计算结构，并添加了在交换环的扩张中执行计算（第 6 节）以及与计算机代数系统交互（第 7 节）的策略（tactics）。在可能的情况下，我们的目标是在比莫德爾方程所需更一般的层面上工作。我们的形式化解决了几个值得注意的困难，包括：在相同的二次环设置中发展一般理论并计算特定数值，通过类型类系统中的计算解决侧面条件问题（第 3.1 和 3.2 节），并通过显式计算类群的方法避免更高级的数的几何（第 4.1 节）。我们希望这将为（计算）代数数论中的进一步形式化发展铺平道路，特别是许多其他有趣的丢番图方程的显式求解。

全局域的类群的有限性之前已被形式化 [3]，作为 mathlib 库 [30] 的一部分（使用 Lean 定理证明器的第 3 版 [8]）。在此基础上，我们显式计算了几个二次数域的类群作为我们工作的一部分，并充实了周围的理论。我们形式化的完整源代码可在线获取。¹

2 Mathematical background

我们用来研究丢番图方程解（尤其是证明无解）的基本工具之一是下降法。主要思想是假设存在一个解，并选择它在某种意义上是最小的。然后，变换这个解以“下降”到方程的另一个解，该解比前一个解更小，从而与其最小性相矛盾。如果变换可以无条件地应用，则方程没有解；否则，如果方程有解，它将对解产生约束条件。

费马曾用这种方法证明了他著名的费马大定理的一个实例，即方程

$$x^4 + y^4 = z^4 \quad (2)$$

没有非零整数解。更准确地说，费马在丢番图的《算术》笔记中证明了 ([14, Section 1.6]) 不存在面积为平方数的直角三角形。这可以重新表述为方程 $a^4 - b^4 = c^2$ 不存在非零整数解，由此可推导出 (2) 没有非零解。

下降法中的关键步骤是应用以下性质：如果 a, b, c 是三个整数， n 是一个自然数，使得 $\gcd(a, b) = 1$ 且 $ab = c^n$ ，那么存在 $c_1, c_2 \in \mathbb{Z}$ 使得 $a = \pm c_1^n$ 且 $b = \pm c_2^n$ 。然而，在更

*Authors listed in alphabetical order

¹<https://github.com/lean-forward/class-group-and-mordell-equation>

大的环上（例如二次域的整数环）进行下降通常比在 $\mathbb{Z}$ 上更方便。

现在我们回顾一些有用的定义。我们假设读者熟悉基本的环和域理论，这在许多（本科）教材中都有描述，包括 [12, 16]。数域 K 是域 $\mathbb{Q}$ 的有限扩张。它是 $\mathbb{Q}$ 上的有限维向量空间，其维数称为 K （在 $\mathbb{Q}$ 上）的度数（degree）。数域的例子包括 $\mathbb{Q}$ 本身（度数为 1）， $\mathbb{Q}(\sqrt{5}) = \{a+b\sqrt{5} : a, b \in \mathbb{Q}\}$ 和 $\mathbb{Q}(\sqrt{-5}) = \{a+b\sqrt{-5} : a, b \in \mathbb{Q}\}$ （两者的度数均为 2），以及 $\mathbb{Q}(\sqrt[4]{17}) = \{a+b\sqrt[4]{17}+c\sqrt[4]{17}^2+d\sqrt[4]{17}^3 : a, b, c, d \in \mathbb{Q}\}$ （度数为 4）。我们主要使用二次域，即度数为 2 的数域（第 3 节），如上面第二和第三个例子所示。一般而言，任何二次域都同构于形式为 $\mathbb{Q}(\sqrt{d}) = \{a+b\sqrt{d} : a, b \in \mathbb{Q}\}$ 的域，其中整数 $d \neq 1$ 且无平方因子（即不被素数 p^2 整除）。换句话说，这些二次域是通过添加多项式 $X^2 - d$ 的（复）根 $\alpha = \sqrt{d}$ 构造的，在对 d 施加的条件下，该多项式在 $\mathbb{Q}$ 上是不可约的。这可以概括为任意正整数度数 $n \in \mathbb{Z}_{>0}$ ，如下所示。对于在 $\mathbb{Q}$ 上不可约的 n 次多项式，将其任何一个（复）根 α 添加到 $\mathbb{Q}$ 中，将得到度数为 n 的数域 $\mathbb{Q}(\alpha) = \{a_0 + a_1\alpha + \dots + a_{n-1}\alpha^{n-1} : a_0, a_1, \dots, a_{n-1} \in \mathbb{Q}\}$ （对 α 的 n 个不同选择都将产生同构的域）。反过来，任何度数为 n 的数域都将同构于这样的域 $\mathbb{Q}(\alpha)$ 。

从代数和算术的角度来看，（有理）整数 $\mathbb{Z}$ 在其分式域（有理数 $\mathbb{Q}$ ）中构成了一个特别“好”的子环。当从 $\mathbb{Q}$ 推广到任意数域 K 时， $\mathbb{Z}$ 的类似物是数域 K 的整数环，记为 $\mathcal{O}_K$ 。它被定义为 $\mathbb{Z}$ 在 K 中的整闭包：

$$\mathcal{O}_K := \{x \in K : \exists f \in \mathbb{Z}[X] \text{ 为首一多项式, 使得 } f(x) = 0\},$$

其中我们回顾，如果（单变量）多项式的首项系数等于 1，则称其为首一多项式。 $\mathcal{O}_K$ 确实是一个环的事实可以从整闭包的一般性质中得出（[25, Section I.2]）。

一些整数环的例子如下。 $\mathbb{Q}$ 的整数环确实是 $\mathbb{Z}$ 。对于 $K = \mathbb{Q}(\sqrt{d})$ ，像 $d \in \{-5, -2, -1, 2, 3\}$ 这样的值直接给出 $\mathcal{O}_K = \mathbb{Z}[\sqrt{d}] = \{a + b\sqrt{d} : a, b \in \mathbb{Z}\}$ ，但是某些其他 d 值，例如 $d \in \{-3, 5\}$ ，给出 $\mathcal{O}_K = \mathbb{Z}[\frac{1}{2}(1 + \sqrt{d})] = \{a + b\frac{1}{2}(1 + \sqrt{d}) : a, b \in \mathbb{Z}\}$ 。例如， $\mathbb{Q}(\sqrt{-3})$ 的整数环严格大于 $\mathbb{Z}[\sqrt{-3}]$ 这一事实很明显，因为 $\frac{1}{2}(1 + \sqrt{-3})$ （它不是 $\mathbb{Z}[\sqrt{-3}]$ 的元素）是具有整数系数的首一多项式 $X^2 - X + 1$ 的根。最后，对于 $K = \mathbb{Q}(\sqrt[4]{17})$ ，我们有 $\mathcal{O}_K = \{a + b\sqrt[4]{17} + c\frac{1}{2}(\sqrt[4]{17}^2 - 1) + d\frac{1}{4}(\sqrt[4]{17}^3 + \sqrt[4]{17}^2 + \sqrt[4]{17} + 1) : a, b, c, d \in \mathbb{Z}\}$ ，它不能写成对于任何 $\alpha \in \mathcal{O}_K$ 的 $\mathbb{Z}[\alpha]$ 形式，这说明确定整数环可能会变得相当复杂。

前面提到的“好”的属性可以通过以下陈述来精确：数域 K 的整数环 $\mathcal{O}_K$ 是一个所谓的戴德金整环（Dedekind domain）[3, Section 2]。虽然 $\mathcal{O}_K$ 不一定是唯一分解整环（UFD），但 $\mathcal{O}_K$ （或更一般地说，任何戴德金整环）中的非零理想可以唯一地（除了因子的顺序外）分解为（必然是非零的）素理想。

当 D 是一个 UFD（将 ± 1 替换为 D 中的一个单位）时，下降法中的关键含义通常仍然成立，例如当 $D = \mathbb{Z}[i]$ 是高斯整数时，但当 D 不是 UFD 时，例如当 $D = \mathbb{Z}[\sqrt{-5}]$ 时，则不再适用。然而，当 D 是像数域的整数环这样的戴德金整环时，我们对 D 中的理想 I, J, L 有一个下降法：

即对于任何自然数 n ，以下含义成立：

$$\gcd(I, J) = 1 \wedge IJ = L^n \implies I = L_1^n \wedge J = L_2^n \quad (3)$$

对于 D 中的某些理想 L_1, L_2 。

为了从理想回到元素， D 的类群（记为 $\text{Cl}(D)$ ）至关重要。回想一下， D 的分式理想 I 是 K （ D 的分式域）的 D -子模，形式为 $I = xJ$ ，其中 $x \in K$ 且 J 是 D 的理想。设置 $x = 1$ 表明每个理想 $J \subseteq D$ 也是分式理想；为了清楚起见，我们将这种情况称为整理理想。类群定义为 D 的非零分式理想模以形式为 $\langle \alpha \rangle$ 的非零分式理想的等价类群，以理想乘法作为群运算。具体地说，如果存在一个非零 $\alpha \in K$ 使得 $I = \langle \alpha \rangle J$ [3, Section 2]，我们说分式理想 $I \sim J$ ，其中我们用 $\langle S \rangle$ 表示由集合 S 生成的分式理想。特别地， D 的非零（分式或整）理想 I 代表 D 的类群中的单位元，当且仅当 I 是主理想。我们注意到 D 的类群中的每个等价类都有一个整理理想代表它，并且类群的等价定义可以通过对 D 的非零整理理想的乘法幺半群采取适当的等价类（ $I \sim J \iff aI = bJ$ ，对于某个非零 $a, b \in D$ ）给出。

我们考虑手头的情况，其中 D 是数域的整数环，对此成立一个基本定理， D 的类群是有限的，该定理由 Baanen、Dahmen、Narayanan 和 Nuccio 形式化 [3]。 D 的类群的阶，称为 D 的类数，记为 $h(D) := \#\text{Cl}(D)$ ，是衡量 D 中唯一分解失败的指标。特别是， $h(D) = 1$ （即类群 $\text{Cl}(D)$ 是平凡的）当且仅当 D 是 UFD。例如 $h(\mathbb{Z}) = h(\mathbb{Z}[i]) = 1$ 。事实证明 $\mathbb{Z}[\sqrt{-5}]$ 不是一个 UFD，因为例如 $I := \langle 1 + \sqrt{-5}, 2 \rangle$ 是一个非主理想，事实上 $\mathbb{Z}[\sqrt{-5}]$ 的任何非主（分式）理想都等价于 I 。因此 $\text{Cl}(\mathbb{Z}[\sqrt{-5}])$ 的一组完整代表系由 $\{1, I\}$ 给出，并且 $h(\mathbb{Z}[\sqrt{-5}]) = 2$ 。

解决莫德尔方程所需的一个重要属性（这源自类群的定义和群论中的拉格朗日定理）是，对于 D 中的理想 I ：

$$\gcd(n, h(D)) = 1 \wedge I^n \text{ 是主理想} \implies I \text{ 是主理想} \quad (4)$$

最后，我们注意到数域 K 唯一决定了它的整数环 D ，因此也决定了 $\text{Cl}(D)$ 和 $h(D)$ 。因此，也使用术语 K 的类群，记为 $\text{Cl}(K) := \text{Cl}(D)$ ，以及 K 的类数，记为 $h(K) := h(D)$ 。

迫不及待想在莫德尔方程 (1) 的显式上下文中看到上述应用读者，可以在继续下一节之前，直接跳到定理 5.1 及其证明梗概。

3 Quadratic rings and fields

对莫德尔方程的代数求解至关重要，能够方便地处理如 $\mathbb{Q}(\sqrt{-7})$ 和 $\mathbb{Z}[\frac{1}{2}(1 + \sqrt{5})]$ 这样的数域和数环，它们通过向环中添加二次多项式的根给出。有几种构造可以使二次环的概念变得精确。将多项式 $f \in R[X]$ 的根添加到整环 R 的一种方法是取 R 的（分式域的）代数闭（域）扩张 K ，它根据定义包含 f 的根 α ，并将 $R[\alpha]$ 设置为 K 中除了 α 之外还包含 R 的每个元素的最小子环。特别地，这对于数环和数域非常有效，因为在这种情况下，我们总是可以将 K 设置为复数。或者，我们可以通过将 $R[\alpha]$ 设置为商环 $R[X]/\langle f \rangle$ 来将根 α 添加到 R ，其中 α 是 $X \bmod f$ 的等价类。这两种构造以前在 mathlib 中都可用并被使用，在整个库的不同设置中都有应用。

此外, `mathlib` 包含用于环 $\mathbb{Z}[\sqrt{d}]$ 的结构 `zsqrtd`, 它用于定义高斯整数, 并证明一些与佩尔 (Pell) 方程相关的结果。这有一个缺点, 即人们只能将整数的平方根添加到整数中, 因此不能使用此结构形成二次域或环 $\mathbb{Z}[\frac{1}{2}(1 + \sqrt{5})]$ 。

上面描述的一般方法对于我们的目的来说也有同样的两个缺点。技术上的反对意见是, 新环 $R[\alpha]$ 的定义本质上使用了 R 上的环结构, 包括类型 $R[\alpha]$ 中的环结构定义。由于 `Lean` 的合一算法 (unification algorithm) 在复杂类型上表现不佳, 我们希望尽可能避免在类型中进行大型定义。更严重的反对意见是, 这两种构造都不适合进行高效计算。通常, `mathlib` 在定义数学对象时很少考虑可计算性: 证明中的计算是在策略执行期间通过构造证明项来执行的。尽管如此, 策略仍然需要一些有用的数据结构来执行其任务, 并需要一种将表达式反映到此数据类型中的方法。因为复数上没有非平凡的判定过程, 如果我们将 $R[\alpha]$ 定义为复数的子环, 我们就不能轻易地对元素执行计算。如果我们将 $R[\alpha]$ 定义为商环 $R[X]/\langle f \rangle$, 通过选择每个等价类的代表元是可以进行计算的。但是, 使用 $R[X]/\langle f \rangle$ 进行计算是不切实际的, 因为 `Lean` 不知道它可以降低多项式的次数以获得更简单的代表元。此外, `mathlib` 通常不在其多项式环的实现中提供有效的算法, 这意味着无论如何我们都无法使用多项式的当前实现来原生执行计算, 我们必须在策略中重新实现多项式运算。

相反, 对于我们的项目, 遵循既定模式 (类似于在 `mathlib` 中定义复数或四元数代数), 我们引入了一种新的、更专门的方法来将二次多项式的根添加到环中, 我们将其命名为 `quad_ring`。

```
structure quad_ring (R : Type*) (a b : R) :=
  (b1 : R) (b2 : R)
```

上面的代码定义了一个新的类型构造器 `quad_ring`, 使得 `quad_ring R a b` 的元素是两个元素 `b1 b2 : R` 的元组, 其中 R 是一个交换环 (实际上, 交换半环也可以)。元素 `(b1, b2)` 代表元素 $b_1 + b_2\alpha \in R[\alpha]$, 其中 α 满足代数关系 $\alpha^2 = a\alpha + b$ 。我们定义了到 `quad_ring` 的强制转换 (coercion), 允许我们将元素 $x : R$ 隐式地也视为 `quad_ring R a b` 的元素。

该类型实际上代表环 $R[\alpha]$ 的事实可由环结构的定义得出, 省略一些先前的实例, 它看起来像这样:

```
instance : comm_ring (quad_ring R a b) :=
{ add := λ z w, ⟨z.b1 + w.b1, z.b2 + w.b2⟩,
  mul := ⟨λ z w, ⟨z.1 * w.1 + z.2 * w.2 * b,
    z.2 * w.1 + z.1 * w.2 + z.2 * w.2 * a⟩⟩,
  neg := has_neg.neg, sub := has_sub.sub,
  one := 1, zero := 0,
  nsmul := (•), npow := npow_rec,
  nat_cast := coe, int_cast := coe, zsmul := (•),
  .. sorry } -- proofs omitted
```

我们注意到, 在整篇论文中, 我们仅为了解释而省略了证明; 这里描述的所有内容在我们的工作中都已完全形式化 (即没有 `sorry`)。

除了加法和乘法等基本环运算符外, 我们还定义了与 N 和 Z 的标量乘法, 以及从 N 和 Z 到任何 `quad_ring` 的强制转换。我们必须定义这些映射可能会令人惊讶, 因为实际上我们可以为所有环 R 一般地定义一个唯一的环同态 $\mathbb{Z} \rightarrow R$, 这此外还会产生唯一的 $\mathbb{Z}$ -模结构。尽管我们可以证明这些构造是唯一的, 但 `Lean` 并不自动认为它们定义相等。`mathlib` [1, 2] 使用的遗忘继承 (forgetful inheritance) 模式规定结构继承, 例如从 `comm_ring` (`quad_ring R a b`) 继承到 `module Z` (`quad_ring R a b`), 不允许定义新数据。相反, 像标量乘法这样的运算必须从父类原样复制, 在这种情况下是 `comm_ring` 中的 `zsmul` 字段。通过在 `comm_ring` 实例中自己指定运算, 我们获得了对这些定义相等的控制。

3.1 Inclusion of $\mathbb{Z}[\alpha]$ into $\mathbb{Q}(\alpha)$

在二次域 $\mathbb{Q}(\alpha) = \mathbb{Q}[x]/\langle f \rangle$ 中, 我们有一个子环 $\mathbb{Z}[\alpha] = \mathbb{Z}[x]/\langle f \rangle$ 。环 R 的子环 A 的非正式描述是 R 的在环运算下封闭的子集。这里我们将 $A = \mathbb{Z}[\alpha]$ 和 $R = \mathbb{Q}(\alpha)$ 定义为两个不同环的商, 因此 A 不能以直接的方式成为 R 的子集; 在 `Lean` 中, 这将对对应于两种不同的类型, 而不是类型 R 和项 $A : \text{set } R$ 。以类似的方式, 整数 $\mathbb{Z}$ 非正式地是有理数 $\mathbb{Q}$ 的子环, 但形式上不是子集。`mathlib` 相反倾向于将子环关系转化为单射映射: 参数 $(f : A \rightarrow^+ R)$ (`hf : injective f`), 其中 f 是环同态, `hf` 携带单射性的证明 [3]。在环运算下封闭的子集 A 通过使用忘记 $x \in A$ 的强制映射 `(!)` 来适应此模式。

此外, 给定环 A, R , 通常有此包含映射 $A \rightarrow R$ 的规范选择。具有映射到 R 的规范映射的假设由参数 `[algebra A R]` 表示, 其中 `algebra (A R : Type*) [comm_semiring A] [semiring R] : Type*` 是取决于两个类型 A, R 的多参数类型类。这提供了一个规范同态 `algebra_map A R : A →+ R`; 因此子环可以由一对参数 `[algebra A R] (h : injective (algebra_map A R))` 表示。

这里, 方括号表示一个实例参数 (instance parameter), 一种隐式参数形式 [2]。`Lean` 的推导器 (elaborator) 通过依次考虑标记为 `instance` 的声明, 自动合成这些参数的值, 将其类型与参数类型合并并使用匹配的实例。实例本身也可以有实例参数, 这些参数以相同的方式合成。在 `Lean 3` 中, 合成实现为深度优先搜索。这种搜索允许我们在合成期间以类似 `Prolog` 的风格执行非平凡的计算, 如下所述。

在二次环的情况下, `algebra` 实例朴素地将具有类型 `algebra (quad_ring Z a b) (quad_ring Q a b)`。但是, 这包含类型错误, 因为表达式 `quad_ring Z a b` 中 `a b : Z`, 而表达式 `quad_ring Q a b` 中 `a b : Q`。因此, 我们的第一次尝试是通过将类型 $\mathbb{Z} \rightarrow \mathbb{Q}$ 的规范包含映射应用于 `a b` 来定义实例, 或者更一般地对于任何 A -代数 R :

```
instance algebra₁ [algebra A R] (a b : A) :
  algebra (quad_ring Z a b) (quad_ring R
    (algebra_map A R a)
    (algebra_map A R b))
```


表达式 `quad_ring R (algebra_map A R a) (algebra_map A R b)` 的冗长令人不满意，当系数是数字表达式时，情况会变得更糟： $\mathbb{Z}[\sqrt{5}]$ 在 $\mathbb{Q}[\sqrt{5}]$ 中的包含关系记为 `algebra_map (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  0))`

(`algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  5`)。更糟糕的是，对于 `quad_ring  $\mathbb{Q}$  0 d` 的任何结果，都需要在应用到 `quad_ring  $\mathbb{Q}$  (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  0) (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  5)` 之前跨等式 `algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  0 = 0` 传输。

为了开发一个没有这些缺陷的实例，我们将系数 a' $b' : R$ 作为单独的参数添加到 `algebra` 实例中，连同一对假设，断言 a' b' 分别是 a b 的像，确保映射保持良定义：

```
def algebra₂ [algebra A R] (a b : A) (a' b' : R)
  (ha : algebra_map A R a = a')
  (hb : algebra_map A R b = b') :
  algebra (quad_ring A a b) (quad_ring R a' b')
```

对于 a b 的数值，由 `simp` 策略调用的 Lean 简化程序足以证明等式假设，这意味着我们可以定义如下实例：

```
instance algebra_sqrt_5 :
  algebra (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  0 5) :=
  algebra₂  $\mathbb{Z}$   $\mathbb{Q}$  0 5 0 5 (by simp) (by simp)
```

但是，我们不能在全局范围内将 `algebra₂` 声明为 `instance`，因为类型类系统无法自动为 `ha` `hb` 参数提供值。

通常，为了能够推断出实例，实例的所有参数必须要么出现在目标的类型中，要么本身就是实例参数。为了将相等性假设 `ha` `hb` 提升为实例参数，我们将其包装在 `fact` 类中 [2]。这是 `mathlib` 中使用的一个小型包装器类，允许我们将某些证明注册为实例参数：

```
class fact (p : Prop) : Prop := (out [] : p)
```

使用这个，我们的代数实例变成（回想一下实例参数在 [方括号] 中传递）

```
instance algebra₃ [algebra A R]
  (a b : A) (a' b' : R)
  [fact (algebra_map A R a = a')]
  [fact (algebra_map A R b = b')] :
  algebra (quad_ring A a b) (quad_ring R a' b')
```

Lean 富有表现力的类型类系统加上类似 Prolog 的合成允许它执行计算。这意味着我们可以定义一组实例，通过求值自动合成等式证明的 `fact` 实例，或者从不同的角度来看，我们为形式为 `algebra_map A R a = a'` 的表达式提供了大步操作语义。立即终止的计算步骤对应于没有进一步实例参数的实例：

```
instance fact_eq_refl (a : R) : fact (a = a)
instance fact_map_zero (f : A →+* R) :
  fact (f 0 = 0)
instance fact_map_one (f : A →+* R) :
  fact (f 1 = 1)
```

未停止的计算步骤采用一个表示仍需要完成的计算的实例参数，例如下面的 `h` 参数：

```
instance fact_map_neg (f : A →+* R)
  (a : A) (a' : R) [h : fact (f a = a')] :
  fact (f (-a) = -a')
```

最后，为了处理数字参数，我们添加了两个实例，用于计算这些数字的二进制表示，由 `bit0` 和 `bit1` 函数给出：

```
instance fact_map_bit0 (f : A →+* R)
  (a : A) (a' : R) [fact (f a = a')] :
  fact (f (bit0 a) = bit0 a')
instance fact_map_bit1 (f : A →+* R)
  (a : A) (a' : R) [fact (f a = a')] :
  fact (f (bit1 a) = bit1 a')
```

`algebra₃` 实例和一组 `fact` 的组合足以自动合成所需的实例：

```
instance algebra_sqrt_5' :
  algebra (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  0 5) :=
  by apply_instance
```

尽管 Lean 3 的实例合成算法缺乏 Prolog 实现中的许多改进，或者确实缺乏 Lean 4 合成算法中的改进 [28]，我们发现它足够强大，可以在合理的时间内执行这组有限的计算：由于缓存，影响仅限于每次包含形式为 `algebra (quad_ring  $\mathbb{Z}$  0 5) (quad_ring  $\mathbb{Q}$  0 5)` 的目标的声明约 100 毫秒的一次性成本。对于较大的数字，成本也保持在较低水平，因为合成复杂度与二进制表示的大小呈线性关系。我们不需要调整实例或进行其他优化来在我们的项目中使用此设计。

3.2 Rings of integers and Dedekind domains

在建立了基本理论之后，我们专注于形式为 $R[\sqrt{d}]$ 的环，特别是 $\mathbb{Z}[\sqrt{d}]$ 和 $\mathbb{Q}(\sqrt{d}) = \mathbb{Q}[\sqrt{d}]$ （我们也将使用符号 $\mathbb{Z}[\sqrt{d}]$ 和 $\mathbb{Q}(\sqrt{d})$ 在代码示例中表示这些特定环）。对于域 K ，我们证明如果 d 不是完全平方数，则 $K[\sqrt{d}]$ 是一个域（否则，该环将包含零因子）。我们首先将域结构作为定义提供，而不是实例，因为类型类系统并未设置为推断给定的 d 不是完全平方数。

```
def field_of_not_square {K : Type*} [field K]
  {d : K} (hd : ¬ is_square d) :
  field (quad_ring K 0 d)
```

由于在每个需要的地方手动提供域结构会很很不方便，我们使用 `fact` 构造将命题提供给类型类系统，所以我们可以定义一个 `field` 实例：

```
instance {K : Type*} [field K]
  {d : K} [hd : fact (¬ is_square d)] :
  field (quad_ring K 0 d) :=
  field_of_not_square (fact.out hd)
```

为了进一步证明 $\mathbb{Q}(\sqrt{d})$ 是 $\mathbb{Z}[\sqrt{d}]$ 的分式域，我们再次必须小心区分数字 $d : \mathbb{Z}$ 和它的像 $d' : \mathbb{Q}$ ，我们通过使用两个独立的变量并添加一个假设 `[hdd : fact (algebra_map  $\mathbb{Z}$   $\mathbb{Q}$  d = d')]` 来实现这一点。我们通过注册一个类型类实例 `is_fraction_ring R K` 来表达 K 是 R 的分式域，这意味着 K 是包含 R 的最小域；这一事实的标准证明很容易形式化。

最后，我们假设 d 无平方因子并且证明，如果 $d \equiv 2$ 或 $d \equiv 3 \pmod{4}$ ， $\mathbb{Z}[\sqrt{d}]$ 是戴德金整环，并且如果 $d \equiv 1 \pmod{4}$ ， $\mathbb{Z}[\alpha] = \mathbb{Z}[\frac{1}{2}(1 + \sqrt{d})]$ 是戴德金整环，通过证明在给定条件下它们是 $\mathbb{Q}(\sqrt{d})$ 的整数环来实现。正如条件所暗示的，这一事实的证明涉及在环 $\mathbb{Z}/4\mathbb{Z}$ 中工作。这些条件提供了一个很好的例子，说明了处理强制转换时涉及的微妙之处：例如我们可以将条件写为 $d \% 4 = 3$ ，留在 $\mathbb{Z}$ 中并允许直接计算来验证条件，或者我们可以写 $(\uparrow d : \mathbb{Z} \bmod 4) = 3$ ，将 d 转换为 $\mathbb{Z}/4\mathbb{Z}$ 的元素，这在 $\mathbb{Z} \bmod 4$ 中工作时直接适用。我们选择将此条件表述为 $d \% 4 = 3$ ，因为现有的证明自动化可以更容易地检查此条件；每当 d 在证明中被转换为 $\mathbb{Z}/4\mathbb{Z}$ 时，我们可以使用 `norm_cast` 策略 [19] 证明 $3 : \mathbb{Z}$ 强制转换为 $3 : \mathbb{Z} \bmod 4$ 。

对于 $d \equiv 1 \pmod{4}$ 的情况，我们改为选择 $m : \mathbb{Z}$ 使得 $d = 4m + 1$ ，并将 $\mathbb{Z}[\alpha]$ 写成 `quad_ring 1 m`。同样，我们提供了一个 `is_fraction_ring` 实例，这次在假设 `[hdm : fact (d' = 4 * ↑m + 1)]` 下。尽管此实例可以通过将强制转换 $\mathbb{Z} \rightarrow \mathbb{Q}$ 替换为 `algebra_map R K` 来泛化，但我们不认为为了这种普遍性而牺牲使用强制转换的便利性是值得的。

所讨论的环是戴德金整环的事实是作为 $\mathbb{Q}(\sqrt{d})$ 的整数环的推论。换句话说，我们必须证明元素 $x \in \mathbb{Q}(\sqrt{d})$ 是 $\mathbb{Z}[\sqrt{d}]$ 或 $\mathbb{Z}[\alpha]$ 的元素，当且仅当 x 是整的：它是某个具有整数系数的首一多项式的根。由于 $\sqrt{d}$ 、 α 和每个整数都满足此属性，并且整性在加法和乘法下保持不变，我们可以很容易地证明 $\mathbb{Z}[\sqrt{d}]$ 或 $\mathbb{Z}[\alpha]$ 的每个元素都是整的。我们通过显式计算得出 $a + b\sqrt{d}$ 的最小多项式（具有 $\mathbb{Q}$ 中的系数）是 $X^2 - 2aX + a^2 - db^2$ （如果 $b \neq 0$ ）来证明逆命题；如果 $a + b\sqrt{d}$ 是整的，则这些系数是整数，因为 $\mathbb{Z}$ 是分式域为 $\mathbb{Q}$ 的欧几里得整环。如果 d 在模 4 下不是完全平方数（即 $d \in \{2, 3\} \pmod{4}$ ），这意味着 b 是一个整数，由此可得 a 也是一个整数，从而 $a + b\sqrt{d} \in \mathbb{Z}[\sqrt{d}]$ 。如果 $d \equiv 1 \pmod{4}$ ，我们反而可以证明存在整数 x, y 使得 $a = x + \frac{1}{2}y$ 和 $b = \frac{1}{2}y$ ，这样 $a + b\sqrt{d} = x + y\frac{1}{2}(1 + \sqrt{d}) \in \mathbb{Z}[\alpha]$ 。结果是一对定理

```
theorem is_integral_closure_1 {m : ℤ} {d' : ℚ}
  [hdm : fact (d' = 4 * ↑m + 1)] :
  ¬is_square d' → squarefree (4 * m + 1) →
  is_integral_closure (quad_ring ℤ 1 m) ℤ
    ℚ(√d')
theorem is_integral_closure_23 {d : ℤ} {d' : ℚ}
  [hdd' : fact (algebra_map ℤ ℚ d = d')] :
  d % 4 = 2 ∨ d % 4 = 3 → squarefree d →
  is_integral_closure ℤ[√d] ℤ ℚ(√d')
```

为 $\mathbb{Q}(\sqrt{d})$ 的整数环提供了模型。

4 Class group calculations

由 Baanen、Dahmen、Narayanan 和 Nuccio 形式化 [3] 的一个经典结果是数域类群的有限性。我们建立在这个形式化之上来计算一些非平凡的类群。尽管现有的形式化可以通过提供整数环的显式描述来使其有效，但它提供的界限非常弱，会导致不必要的工作，所以我们选择显

式地重新进行计算。一般来说，得到整数环为 $\mathcal{O}_K$ 的数域 K 的类群有两个基本要素。首先，需要得到类群的一个有限生成集。导致这一结果的一个众所周知的经典结果是闵可夫斯基界（Minkowski bound），正如本节末尾简要讨论的那样。然而，我们采用了通过广义带余除法的更直接的方法，如下面小节所述。最后，给定一个有限生成集，需要建立乘法结构以完全确定该群。在我们目前的方法中，我们专注于小型（包括非平凡的）示例。

4.1 Upper bound

我们将建立在 [3, Section 8.2] 中关于数域 K 的类群有限性的形式化基础之上。我们的起点是前述著作中形式化的一个定理的以下推广。

Theorem 4.1. 设 I 为 $\mathcal{O}_K$ 的非零理想，设 M 为非零整数的有限集，并假设对于所有 $a, b \in \mathcal{O}_K$ 且 $b \neq 0$ ，存在 $q \in \mathcal{O}_K$ 和 $r \in M$ 使得

$$|\text{Norm}_{\mathcal{O}_K/\mathbb{Z}}(ra - qb)| < |\text{Norm}_{\mathcal{O}_K/\mathbb{Z}}(b)|. \quad (5)$$

那么存在 $\mathcal{O}_K$ 的一个理想 J 使得

$$I \sim J \text{ 且 } J \left| \left\langle \prod_{m \in M} m \right\rangle \right|.$$

我们注意到，与其采用由 M 中所有元素的乘积生成的理想，我们也可以采用由 M 中元素的任何非零公倍数（例如它们的最小公倍数）生成的理想。但是，当关注生成元时，我们可以使用理想的唯一分解将 J 限制为素理想，因此，通过简单地对 M 中的所有元素求积，我们不会引入任何更多的素理想。我们在 Lean 中形式化了上述陈述，将 $\mathbb{Z}$ 推广到具有绝对值运算 `abv` 的任何欧几里得整环 R ，并将 $\mathcal{O}_K$ 推广到任何戴德金整环 S ，使得 $L = \text{Frac}(S)$ 是 $K = \text{Frac}(R)$ 的有限扩张。

```
theorem exists_mk0_eq_mk0' (I : (ideal S)⁰)
  (M : finset R) :
  (∀ m ∈ M, algebra_map R S m ≠ 0) →
  (∀ (a : S) (b ≠ 0) → (∃ (q : S) (r ∈ M),
    abv (norm R (r * a - q * b)) <
      abv (norm R b))) →
  ∃ (J : (ideal S)⁰), mk0 L I = mk0 L J ∧
    algebra_map _ _ (∏ m in M, m) ∈ J
```

这里 `mk0 L` 是将非零理想 $I : (\text{ideal } S)^0$ 映射到其在类群中等价类的同态。

为了建立定理 4.1 中给出的广义带余除法的假设，将整数环中的范数估计 (5) 转换为分式域中的范数估计，并在两边除以因子 b 是很实用的。

```
lemma exists_lt_norm_iff_exists_le_one
  [is_integral_closure S R L]
  (M : finset R) :
  (∀ (a : S) (b ≠ 0), ∃ (q : S) (r ∈ M),
    abv (norm R (r * a - q * b)) <
      abv (norm R b)) ↔
  (∀ (y : L), ∃ (q : S) (r ∈ M),
    abv (norm K (r * y - q * 1)) < 1)
```

在证明中，我们花了最多精力极其繁琐地证明 b 非零当且仅当其在 L 中的像非零，当且仅当其在 R 上的范数非零，当且仅当其在 K 上的范数非零，以证明两边除以 b 的想法的合理性。

我们现在来看上述结果的具体推论。我们考虑二次数域 $K = \mathbb{Q}(\sqrt{d})$ ，其中 $d \in \mathbb{Z}$ 并且假设 $d \equiv 2$ 或 $3 \pmod{4}$ 并且 d 是无平方因子的（除非另有明确指示），这是在 Lean 中使用类型类假设 `[fact (d % 4 = 2 ∨ d % 4 = 3)]` 和 `[fact (squarefree d)]` 分别指定的，因为它们对于戴德金整环实例本来就是需要的。为了得到 $d \in \{-1, -2, -5, -6\}$ 的类群结果，我们对这样的 $\gamma \in \mathbb{Q}(\sqrt{d})$ 进行必要的范数估计，集合 M 取为 $\{1, 2\}$ ：

```
theorem sqrt_neg.exists_q_r
  [fact (algebra_map ℤ ℚ d = d')]
  (γ : ℚ(√d)) : -6 ≤ d → d ≤ 0 →
  ∃ (q : ℤ[√d]) (r ∈ {1, 2}),
    abs (norm ℚ (r · γ - q · 1)) < 1
```

证明是初等的，基本上包括根据 γ 的有理数分量进行案例区分，并为这些分量建立不等式。在我们为不等式的非线性部分建立估计之后，策略 `linarith` 可以解决这些目标。

将上面的这个定理与前面提到的两个结果结合起来现在表明，对于 $d \in \{-1, -2, -5, -6\}$ ， $\mathbb{Z}[\sqrt{d}]$ 中的任何非零理想 I 在类群中等价于整除 $\langle 1 \rangle$ 或 $\langle 2 \rangle$ 的理想 J 。下一个任务是将 $\langle 2 \rangle$ 分解为素理想。为此，我们显式定义了一个非零理想 `sqrt_2 d`，使得 $(\text{sqrt_2 } d)^2$ 是由 2 生成的理想，并且我们证明了这个理想是不可约的。`sqrt_2 d` 的定义取决于 d 模 4 的余数。

```
def sqrt_2 [fact (d % 4 = 2 ∨ d % 4 = 3)] :
  (ideal ℤ[√d])0 :=
if d % 4 = 2 then
  ⟨ideal.span {(0, 1) : ℤ[√d]}, 2⟩,
  sorry) -- Proof of nonzeroness omitted
else
  ⟨ideal.span {(1, 1) : ℤ[√d]}, 2⟩,
  sorry) -- Proof of nonzeroness omitted
```

我们通过显式证明每个理想的生成元集包含在另一个理想中，来检查 `span {2}` 在这两种情况下都分解为 $(\text{sqrt_2 } d)^2$ 。检查这个等式比基于非正式论证的简单性看起来要困难得多，并且每个等式的形式化占据了 20 多行代码。这是因为有许多常规的整除性和显式的数值条件需要检查，目前这些都必须手工指导。在这个领域，应该开发改进的策略或更好的抽象，以减少在证明助手中进行类似计算的工作量。

为了证明 `sqrt_2 d` 是一个素理想，我们考虑它的范数。在代数数论中，整理理想的范数可以取为整数，即绝对理想范数，或者作为子域的理想，给出相对理想范数。我们将戴德金整环 R 的绝对理想范数形式化为乘法同态，对于 R 的理想 I ，如果商 R/I 是有限的，则定义为 $N(I) = |R/I|$ ，否则为 0。我们分两步证明乘法性：首先我们证明当 I 和 J 互素时 $N(IJ) = N(I)N(J)$ ，然后我们证明当 P 是素理想且 i 是自然数时 $N(P^i) = N(P)^i$ ；戴德

金整环中理想的唯一分解性质允许我们得出结论，所有 I 和 J 都满足 $N(IJ) = N(I)N(J)$ 。

我们计算出 `sqrt_2 d` 的范数是素数 2，并得出结论 `sqrt_2 d` 本身是素的：如果它分解为 IJ ，则 $N(IJ) = N(I)N(J) = 2$ ，这意味着不失一般性 $N(I) = |R/I| = 1$ ，因此 $I = R$ 。

我们得出结论，对于 $d \in \{-1, -2, -5, -6\}$ ， $\mathbb{Z}[\sqrt{d}]$ 的类群由包含 `sqrt_2 d` 的类生成，我们称之为 `class_group.sqrt_2 d`。这个理想平方为 2 的事实表明该类的阶为 1 或 2。特别地，类群最多包含 2 个元素：

```
theorem class_group_eq (d : ℤ)
  [fact (d % 4 = 2 ∨ d % 4 = 3)]
  [fact (squarefree d)]
  (I : class_group ℤ[√d]) :
  -6 ≤ d → d ≤ 0 → I ∈ {1, sqrt_2 d}
```

类似地，但付出了更多的努力，我们证明了 $\mathbb{Z}[\sqrt{-13}]$ 的类群由这两个类生成。我们将集合 $M = \{1, 2, 3, 4\}$ 用于范数估计（ $\{1, 2, 3\}$ 也可以），并求助于库默尔-戴德金定理（Kummer-Dedekind theorem）来证明 $\langle 3 \rangle$ 是 $\mathbb{Z}[\sqrt{-13}]$ 中的素理想。

对于解决本定理涵盖的 d 的莫德尔方程的应用，这实际上足以完成证明，因为对于这些应用，只需要 3 不整除类数。尽管如此，在一般情况下，有必要完全计算类数而不仅仅是 2 的上限，因此我们也形式化了展现类群的不同元素并给出大小的不平凡下界的方法。

4.2 Lower bound

为了完成我们对类群的计算，我们需要确定我们找到的类是否不同。为此，我们需要确定某些理想是否是主理想。虽然我们可以通过给出一个显式的生成元来轻松证明是主理想，但证明非主理想可能更棘手。然而，对于我们的例子，至少当 $d < -2$ 时，我们可以很快在相当大的普遍性下获得非主理想的必要证明。连同已经建立的 $(\text{sqrt_2 } d)^2$ 的主理想性，我们得到：

```
lemma order_of_sqrt2 (d : ℤ)
  [hd : fact (d % 4 = 2 ∨ d % 4 = 3)] [fact
    (squarefree d)] :
  d < -2 → order_of (class_group.sqrt_2 d) = 2
```

为了证明 `sqrt_2 d` 不是主理想，我们取一个假设的生成元 $a + b\sqrt{d}$ 的范数，给出一个方程 $a^2 - db^2 = 2$ ，其中 $a, b \in \mathbb{Z}$ 。一个简单的估计表明，对于 $d < -2$ 不可能有任何解。然后拉格朗日定理暗示：

```
theorem two_dvd_class_number (d : ℤ) (d' : ℚ)
  [fact (algebra_map ℤ ℚ d = d')] [fact (d % 4 =
    2 ∨ d % 4 = 3)] [fact (squarefree d)] :
  d < -2 → 2 ∣ class_number ℚ(√d')
```

这说明对于任何 $d < -2$ 且 $d \equiv 2$ 或 $3 \pmod{4}$ 的无平方因子整数 d ，有 $2 \mid h(\mathbb{Q}(\sqrt{d}))$ 。

最初，`mathlib` 中类群的定义依赖于分式域的选择，这意味着在我们形式化的原始版本中，我们必须保持 $d : \mathbb{Z}$ 和 $d' : \mathbb{Q}$ 之间的区别。这使得我们不得不在 d 和 d' 之间反复传递假设。由于分式域的每种选择都给出了一个

同构的类群，我们重新定义了 `class_group` 以改为使用特定的构造 `fraction_ring` 作为分式域，并在需要其他选择时插入显式同构。这意味着我们只需要在计算类数的最后一步转移到 $d' : \mathbb{Q}$ 。

4.3 Explicit results

对于 $d \in \{-5, -6, -13\}$ ，上界和下界共同意味着 $h(\mathbb{Q}(\sqrt{d})) = 2$ ：

```
lemma class_number_5 : class_number  $\mathbb{Q}(\sqrt{-5}) = 2$ 
lemma class_number_6 : class_number  $\mathbb{Q}(\sqrt{-6}) = 2$ 
lemma class_number_13 : class_number  $\mathbb{Q}(\sqrt{-13}) = 2$ 
```

事实上，我们在这些情况下也得到了类群 $\text{Cl}(\mathbb{Q}(\sqrt{d}))$ 元素的显式（不同）代表元。对于 $d \in \{-1, -2\}$ ，不难证明 $\text{sqrt}_2 d$ 是主理想，导致 $h(\mathbb{Q}(\sqrt{d})) = 1$ ：

```
lemma class_number_1 : class_number  $\mathbb{Q}(\sqrt{-1}) = 1$ 
lemma class_number_2 : class_number  $\mathbb{Q}(\sqrt{-2}) = 1$ 
```

或者，人们可以通过使用 $M = \{1\}$ 进行范数估计更直接地获得这些 UFD 结果。

4.4 Other possible approaches

众所周知，通过所谓的闵可夫斯基界，在 $\mathbb{Q}$ 上度数为 n 的数域 K 的类群由满足以下条件的素理想 P 的类生成

$$\text{Norm}(P) \leq \sqrt{|\Delta_K|} \left(\frac{4}{\pi} \right)^{r_2} \frac{n!}{n^n} \quad (6)$$

其中 r_2 表示 $K \hookrightarrow \mathbb{C}$ 的复数、非实数嵌入的共轭对的数目， Δ_K 表示 K 的判别式（例如见 [25, Section I.5]）。在我们的例子中，对于 $K = \mathbb{Q}(\sqrt{d})$ ，其中 $1 \neq d \in \mathbb{Z}$ 无平方因子，我们有恒等式

$$\Delta_K = \begin{cases} d & \text{如果 } d \equiv 1 \pmod{4}, \\ 4d & \text{如果 } d \equiv 2, 3 \pmod{4}. \end{cases}$$

支持 (6) 证明的主要结果是闵可夫斯基凸体定理，该定理以前在简单类型论和依赖类型论中都被形式化过（例如在 Isabelle/HOL [13] 和 Lean² 中）。

虽然本节概述的方法和闵可夫斯基界都适用于任何度数的数域，但在二次域的设置中，还有另一种计算类群（或至少是类数）的方法。在 $K = \mathbb{Q}(\sqrt{d})$ 的情况下，对于负无平方因子整数 d ，解析类数公式 [25, Section VII.5] 给出了一个类数公式作为显式有限和

$$h(K) = \frac{w}{2\Delta_K} \sum_{a=1}^{|\Delta_K|-1} \left(\frac{\Delta_K}{a} \right) a,$$

其中 w 是 K 的单位数，即 2，除非 $d = -1$ 或 -3 （在这种情况下 w 分别为 4, 6），而 $\left(\frac{\Delta_K}{a} \right)$ 表示雅可比符号（Jacobi symbol）。将此结果形式化用于我们的目的的主要困难在于所需的解析结果与本项目其余部分使用的更多代数结果相去甚远。

5 Explicit solution of Mordell equations

本文的重点是通过计算类群和（理想）下降来形式化解某些莫德方程 (1)。但我们首先简要回顾一下莫德方程本身的历史。

1657 年，费马在一封信中声称， $y^2 = x^3 - 2$ 在正整数范围内的唯一解是 $(x, y) = (3, 5)$ 。1914 年，莫德 [21] 从两个角度研究了一般方程 $y^2 = x^3 + d$ ： d 不存在任何整数解的条件，以及一种基于理想下降寻找某些 d 的解的方法。这个方程很快被称为莫德方程。1918 年，他 [22] 接着证明了一个非构造性的有限性定理：对于每个非零 $d \in \mathbb{Z}$ ，莫德方程有有限个整数解。另见莫德的书 [24, Chapter 26]。1967 年，Baker [4] 证明了以下有效结果：对于每个 $d \in \mathbb{Z}$ ， $d \neq 0$ ，如果 $x, y \in \mathbb{Z}$ 满足 $y^2 = x^3 + d$ ，则

$$\max\{|x|, |y|\} \leq e^{10^{10}|d|^{10000}}. \quad (7)$$

在此基础上，人们获得了渐近更锐利的有效界限。最值得注意的是，Stark [29] 在 1973 年证明，对于任何 $\epsilon > 0$ ，存在一个可有效计算的常数 C_ϵ 使得现在 $\max\{|x|, |y|\} < C_\epsilon^{|d|^{1+\epsilon}}$ 。直接使用这些界限来求解任何给定 d 的莫德方程是不切实际的，但人们已经开发了有效求解莫德方程的技术，例如 Bennett 和 Ghadermanzi [5] 在 2015 年求解了所有（非零） $|d| < 10^7$ 的 (1)。

形式化上述任何有效界限，或形式化上述针对 $|d| < 10^7$ 的解析，都需要首先开发大量的背景理论。作为超越使用初等数论求解丢番图方程的一步，我们使用类群下降形式化了一个灵活的存在性结果，该结果出现在莫德 [21, Section 8] 中。

Theorem 5.1. 设 $d < 0$ 是一个无平方因子的整数，且 $d \equiv 2$ 或 $3 \pmod{4}$ ，并假设 $3 \nmid h(\mathbb{Z}[\sqrt{d}])$ 。则莫德方程 (1) 有解当且仅当 $d = -3m^2 \pm 1$ （对于某个 $m \in \mathbb{N}$ ），在这种情况下 $y = \pm m(3d + m^2)$ 。

现在让我们简要概述一下这个结果的证明：

Proof. 假设我们有莫德方程 (1) 的一个解 (x, y) 。那么通过将 d 移到另一边并在环 $D := \mathbb{Z}[\sqrt{d}]$ 上分解该多项式，我们得到

$$(y + \sqrt{d})(y - \sqrt{d}) = x^3.$$

因此，对于分别由 $y + \sqrt{d}$ 、 $y - \sqrt{d}$ 和 x 生成的理想 I 、 J 和 L ，我们有 $IJ = L^3$ 。在上面的设置中，理想 I 和 J 是互素的，因为它们的最大公约数整除 $2y$ 和 $2\sqrt{d}$ ；然而，可以证明 y 和 d 是互素的且 x 是奇数，所以理想的最大公约数必须是一。因为 D 是数域 $\mathbb{Q}(\sqrt{d})$ 的整数环，我们可以调用 (3)，这特别意味着存在 D 中的理想 L_1 使得 $I = L_1^3$ 。注意，根据构造 I 是主理想，因此结合假设 $3 \nmid h(\mathbb{Z}[\sqrt{d}])$ ，我们由 (4) 得到 L_1 是主理想。这转化为 $y + \sqrt{d} = u\alpha^3$ ，其中 $\alpha \in D$ 且单位 $u \in D^*$ 。由于 D 中的所有单位都是立方，我们可以不失一般性地假设 $u = 1$ 。记 $\alpha = a + b\sqrt{d}$ ，我们得到

$$y + \sqrt{d} = (a + b\sqrt{d})^3 = a(a^2 + 3b^2d) + b(3a^2 + b^2d)\sqrt{d}.$$

²<https://github.com/leanprover-community/mathlib/pull/2819>

比较等式两边 $\sqrt{d}$ 的系数得出

$$1 = b(3a^2 + b^2d).$$

由于这构成了 1 在 (有理) 整数 $\mathbb{Z}$ 中的分解, 我们得到 (代入 $b^2 = 1$)

$$b = 3a^2 + d = \pm 1.$$

相应的 y 值现在可以计算出来, 结果很快就得出了。□

5.1 Formalizing solution of Mordell equations

我们形式化了定理 5.1 的证明, 遵循上述梗概作为 Lean 中的以下陈述:

```
theorem Mordell_d (d : ℤ) (d' : ℚ) (hd : d ≤ -1)
  (hdm : d % 4 = 2 ∨ d % 4 = 3)
  (hdd' : algebra_map ℤ ℚ d = d')
  (hdsq : squarefree d) [fact (¬ is_square d')]
  (hcl : (3 : ℕ).gcd (class_number ℚ(√d')) = 1)
  (x y : ℤ) (h_eqn : y ^ 2 = x ^ 3 + d) :
  ∃ m : ℤ, ∃ y = m * (d * 3 + m ^ 2) ^ 1/2
    (d = 1 - 3 * m ^ 2 ∨ d = -1 - 3 * m ^ 2)
```

请注意, 我们使用 `fact` 向类型类系统提出假设, 如第 3 节中那样。

该证明大致遵循刚才概述的非正式内容。首先, 我们对 $y^2 - d = (y + \sqrt{d})(y - \sqrt{d})$ 这一项进行因式分解, 使用算术策略 `ring` 和 `calc_tac` 检查因式分解。然后我们传递到 `class_group ℤ[√d] ℚ(√d')` 中的一个等式, 并应用理想下降引理 (3) 来获得理想 $\langle y + \sqrt{d} \rangle$ 是一个立方。要检查理想 $\langle y + \sqrt{d} \rangle, \langle y - \sqrt{d} \rangle$ 是互素的这一侧面条件, 需要多行代码, 并且看起来比这些证明的非正式呈现要复杂得多。在这种情况下, 更好的策略或抽象可能会有用。要检查的陈述如下:

```
lemma gcd_lemma {x y d : ℤ} [fact (d % 4 = 2 ∨ d % 4 = 3)] [fact (squarefree d)]
  (h_eqn : y ^ 2 - d = x ^ 3) :
  gcd (span {⟨y, 1⟩}) (span {⟨y, -1⟩}) = 1
```

在纸笔证明中, 作者通常采取逆否命题, 并假设存在一个同时整除 $\langle y + \sqrt{d} \rangle, \langle y - \sqrt{d} \rangle$ 的素理想 $\mathfrak{p}$, 并推导出与 $\mathfrak{p}$ 为素数相矛盾。然而, 对于形式化, 我们发现通过对两个理想的范数施加越来越多的限制来简单地计算它们的 GCD 更为方便。

`gcd_lemma` 的重要要素之一是这样一个事实: 对于满足定理假设的 d , 莫德方程的任何解必定有 x 为奇数。这个结果的最强相关版本实际上是:

```
lemma x_odd_two_three_aux {x y d : ℤ}
  (hd : ∃ d ∈ {2, 3, 5, 6, 7} : finset (zmod 8)))
  (h_eqn : y ^ 2 - d = x ^ 3) : ¬ even x
```

这可以通过推广到关于 `zmod 8` 元素的陈述并应用内置判定程序 `dec_trivial` 检查所有 $8 \times 8 \times 8$ 个情况来证明。注意, 这个引理也适用于 $d \equiv 5 \pmod{8}$, 因此这是这里形式化的方法可以应用的与 $d \not\equiv 2, 3 \pmod{4}$ 时最相似的情况。

由此我们得到一个理想, 其立方为 $\langle y + \sqrt{d} \rangle$, 并且根据对类群的假设, 它是主理想。从主理想的等式传递到

它们生成元在相差一个单位下的等式, 是交换代数中适用于任何整环的非常一般引理的应用。

对于 $d \leq -1$ 且 $d \equiv 2, 3 \pmod{4}$, $\mathbb{Z}[\sqrt{d}]$ 的单位的计算, 可以很快从以下事实得出: 范数

$$\text{Norm}(a + b\sqrt{d}) = a^2 - db^2$$

只有在 a, b, d 为小数值时才能等于 1, 并且可以使用线性算术策略 (如 `linarith`) 处理。在所有情况下, 我们都可以检查单位群的阶是二的幂, 因此所有单位都是立方。

我们得出结论: $\exists z : \mathbb{Z}[\sqrt{d}], z^3 = \langle y, 1 \rangle$ 通过对 z 的分量 m, n 进行分类, 并对 `quad_ring` 应用外延性, 导致了所需的整数等式:

$$\begin{aligned} (m^2 + 3 * d * n^2) * m &= y \\ (d * n^2 + 3 * m^2) * n &= 1 \end{aligned}$$

在因式分解发现 $n = \pm 1$ 后, 它给出了定理陈述中存在量词的见证。

这个定理的逆命题, 即当条件 $d = -3m^2 \pm 1$ 对于某个 m 成立时, 给定的 y 值实际上给出了一个解, 这实际上在更广泛的范围内成立, 而且原因也简单得多。所以我们按如下陈述和证明, 只需检查这两种情况都给出任意交换环上方方程的解:

```
lemma Mordell_contra (d m : ℝ)
  (hd : d = 1 - 3 * m^2 ∨ d = -1 - 3 * m^2) :
  (m * (d * 3 + m^2))^2 = (m^2 - d)^3 + d :=
  by rcases hd with rfl | rfl; ring
```

现在我们来在具体情况中应用 `Mordell_d`。多亏了第 4.3 节中类数的计算, 我们可以无条件地将上述结果应用于 $d \in \{-1, -2, -5, -6, -13\}$ 。这给出了以下定理:

```
theorem Mordell_minus1 (x y : ℤ)
  (h_eqn : y^2 = x^3 - 1) : y = 0
theorem Mordell_minus2 (x y : ℤ)
  (h_eqn : y^2 = x^3 - 2) : y = 5 ∨ y = -5
theorem Mordell_minus13 (x y : ℤ)
  (h_eqn : y^2 = x^3 - 13) : y = 70 ∨ y = -70
theorem Mordell_minus5 (x y : ℤ) : y^2 ≠ x^3 - 5
theorem Mordell_minus6 (x y : ℤ) : y^2 ≠ x^3 - 6
```

注意在前面三种情况中, 方程 $d = -3m^2 \pm 1$ 有解, 分别对应 $m = 0, m = 1$ 和 $m = 2$ 。这些产生了所示的 y 值, 它们确实给出了 (1) 的解。在最后两种情况下不存在解。

在不存在结果和其他 $d = -3m^2 \pm 1$ 无解的情况下, 我们需要证明整数中某些二次方程无解。这简化为检查特定的有理数是否为完全平方数。虽然为其编写一个判定过程并不是很困难, 但我们选择走捷径, 简单地检查必要的二次方程在精心选择的模数 n 下无解。这种方法的优点是环 `zmod n` 已经在 Lean 的 `mathlib` 中被证明具有可判定的相等性并且是有限的, 因此内置的判定程序 `dec_trivial` 无需额外工作即可完成这些目标。这种方法的缺点是用户必须首先手工找到一个模数 n , 使得所考虑的方程无解。

5.2 Elementary and elliptic curve approaches

正如类群计算一样，讨论一些求解（某些）莫德方程的其他可能方法似乎是合适的。最重要的是，它将把形式化求解的莫德方程放在某个视角下。

在没有整数解的情况下，通过使用较不高级的数论工具（如二次互反律、整数的唯一分解和同余式 [11, Chapter XXIII]），有可能给出更简单的无解证明。这些证明更容易形式化，因为技术先决条件较少，但它们不能推广到包含有解的更高级情况。

方程 (1) 在 (x, y) 平面上被视为代数曲线时，定义了一条椭圆曲线。莫德证明了在任何椭圆曲线上具有整数坐标的点（即定义方程的整数解）的数量是有限的 [23]。另一方面，椭圆曲线上的有理点构成一个阿贝尔群，称为该曲线的莫德-韦伊群（Mordell-Weil group），它是有限生成的，通常具有无限阶的生成元。一般来说，确定椭圆曲线的秩是一个未解决的问题，但目前可以在许多情况下有效地计算它。对于莫德曲线，确切地知道何时可能出现挠点（torsion）。因此，从一般角度来看，莫德方程最有趣的情况是那些曲线的秩至少为一的情况，因此有无穷多个有理解，但只有有限多个整数解。

在下表中，对于满足 $-1 \geq d \geq -13$ 的每个整数 d ，我们列出了与对应于 d 的莫德椭圆曲线 $E_d : y^2 = x^3 + d$ 相关的上述数量，即 E_d 的莫德-韦伊群的秩 $\text{rk}(E_d)$ 以及其上整数点的数量 $|E_d(\mathbb{Z})|$ ，以及二次域 $K_d = \mathbb{Q}(\sqrt{d})$ 的类数 $h(K_d)$ 。下表中加粗的元素表示已形式化的结果。此外，非无平方因子 d 的类数 $h(K_d)$ 的计算依赖于同构于由 d 的无平方因子部分的平方根生成的二次域。

$-d$	1	2	3	4	5	6	7	8	9	10	11	12	13
$\text{rk}(E_d)$	0	1	0	1	0	0	1	0	0	0	2	0	1
$ E_d(\mathbb{Z}) $	1	2	0	4	0	0	4	1	0	0	4	0	2
$h(K_d)$	1	1	1	1	2	2	1	1	1	2	1	1	2

对于求解 (1)，当 $|E_d(\mathbb{Z})| \neq 0$ 时，显然不能使用初等的无解证明；当 $\text{rk}(E_d) \neq 0$ 时，不能使用直接的椭圆曲线证明；在我们的方法中，一旦 $h(K_d) \neq 1$ ，类群就是不可避免的。请注意， $d = -13$ 的情况具有所有这三个特征。

6 Computational tactics

为了在 `quad_ring` 内实现计算，我们使用了一对策略，我们称之为 `quad_ring.calc_tac` 和 `times_table_tac`。我们最初使用的是 `calc_tac` 策略，它特定于 `quad_ring` 并且需要更多手动指导。对于更复杂的目标，我们实现了更强大、更通用的策略 `times_table_tac`。

6.1 quad_ring.calc_tac

利用我们的设置，`calc_tac` 策略可以根据现有的 `mathlib` 策略进行极其简短的实现：

```
meta def calc_tac : tactic unit :=
  \[rw quad_ring.ext_iff;
    repeat { split; simp; ring }]
```

首先，它使用外延性引理 `quad_ring.ext_iff` 将等式 $x = y$ （其中 $x, y : \text{quad_ring } R \ a \ b$ ）转换为合取 $x.b1 = y.b1 \wedge x.b2 = y.b2$ 。然后 `split` 策略将包含合取的单

个目标替换为合取两侧的两个目标。`simp` 策略调用简化程序重写目标。最后，`ring` 策略（基于同名的 Coq 策略 [15]）通过以霍纳范式（Horner normal form）重写等式两边来求解多项式表达式的等式。因此，`calc_tac` 可以解决由 `quad_ring` 中的等式组成的任何目标，前提是简化程序可以将目标简化为交换（半）环中多项式表达式的分量等式。

例如，为了实例化 `comm_ring (quad_ring R a b)` 结构，我们必须为字段 `mul_comm (x y : quad_ring R a b) : x * y = y * x` 提供一个值。

我们通过单次调用 `calc_tac` 来实现。在应用外延性并拆分合取后，策略产生两个目标 $(x * y).b1 = (y * x).b1$ 和 $(x * y).b2 = (y * x).b2$ 。我们之前已经注册了指定乘积组件的 `@[simp]` 引理：

```
@[simp] lemma mul_b1 (z w : quad_ring F a b) :
  (z * w).b1 = z.b1 * w.b1 + z.b2 * w.b2 * b :=
  rfl
@[simp] lemma mul_b2 (z w : quad_ring F a b) :
  (z * w).b2 = z.b2 * w.b1 + z.b1 * w.b2 +
  z.b2 * w.b2 * a := rfl
```

当简化程序遍历目标的子表达式时，从叶子节点开始，它使用子表达式的头部符号来查找所有相应的带有 `@[simp]` 标记的引理，直到匹配成功。它执行重写并在新的子表达式中重新开始遍历，直到整个表达式被穷尽地重写。在这种情况下，简化程序对 `mul_b1` 和 `mul_b2` 各应用两次：一次在两个目标的左侧，一次在右侧。结果是两个目标 $x.b1 * y.b1 + x.b2 * y.b2 * b = y.b1 * x.b1 + y.b2 * x.b2 * b$ 和 $x.b2 * y.b1 + x.b1 * y.b2 + x.b2 * y.b2 * a = y.b2 * x.b1 + y.b1 * x.b2 + y.b2 * x.b2 * a$ 。最后，在两个目标上调用 `ring` 策略，将每一侧视为变量 $x.b1, y.b1, x.b2, y.b2, a, b$ 中的多元多项式，并利用环 R 的交换性将两边重写为相等的多项式，从而完成证明。

显然，`calc_tac` 的适用性从根本上取决于简化程序将 `quad_ring` 元素的组件简化为多项式表达式的能力。换句话说，为了能够应用 `calc_tac`，用户必须整理一组足够强大的 `@[simp]` 引理，以表达除多元多项式之外的所有计算。因此，我们在 `quad_ring` 中为每个定义配备了一对指定其 `b1` 和 `b2` 分量的引理；这通常可以通过将 `@[sims]` 属性应用于定义来自动化。另一方面，简化程序必须尝试所有引理，直到找到适用的引理，因为它除了头部符号之外没有对集合进行任何索引，因此随着 `@[sims]` 引理集的增长，它会明显变慢。这种担忧是一个相关因素但并非压倒一切，特别是考虑到 Lean 4 承诺使用判别树提供更高效率的简化程序 [9]。

`calc_tac` 结构的另一个缺点是可能在简化阶段累积非常大的表达式。例如，`ring` 策略在目标 $(x - y) * (x^2 + x * y + y^2) = x^3 - y^3$ 上花费几十毫秒，而 `calc_tac` 需要几秒钟才能在 `quad_ring` 上证明相同的等式，因为它首先必须将每个乘法展开为乘法之和，给出包含 72 次变量出现的中间结果。由于这些限制，我们只能将 `calc_tac` 应用于简单的方程，例如 `quad_ring` 的环

公理。为了以更原则性的方式处理更复杂的方程，我们开发了一个更强大策略的原型，称为 `times_table_tac`。

6.2 Times tables

`times_table_tac` 的目标是标准化交换环有限扩张（包括但不限于 `quad_ring`）中的表达式。更准确地说，设 S/R 是交换环的扩张，使得 S 是作为一个自由且有限生成的 R -模，给定 R 中表达式的标准化过程，那么 `times_table_tac` 在多元多项式环 $S[X_1, \dots, X_n]$ 中标准化表达式。

适合 `times_table_tac` 的有限环扩张在数学中很常见，对我们的论文特别相关的一个例子是 $S = R[\alpha]$ ，其中 α 是首一多项式的根，因为 `quad_ring` 是 $R[\alpha]$ 的一个特例，其中 α 的度数为二。满足 `times_table_tac` 所需前提条件的环扩张的其他常见例子包括实数上的复数，给定环 R 上的所有矩阵环 $M_n(R)$ ，以及 $\mathbb{Z}/p\mathbb{Z}$ 上的所有有限域 $GF(p^k)$ 。

我们可以通过将每个 $x \in S$ 写成一组基向量 b 的 R -线性组合来定义 S 的范式；如果 S 是自由 R -模，则这种线性组合的系数是良定义且唯一的。这些线性组合的加法和标量乘法可以逐个分量进行，但是对于 S 中两个元素的乘法，我们需要额外的信息。因为 R 的环扩张 S 也是 R -代数， S 中的乘法是一个 R -双线性映射，如果我们给定 S 作为一个 R -模的有限基 b （由于我们假设 S 是自由且有限的，因此它存在），那么我们可以通过给出每对基元素 b_i, b_j 的乘积 $T_{i,j} \in S$ 来指定乘法运算。因此，为了将 S 中的两个元素 xy 相乘，我们可以将每个元素写为基元素的线性组合 $x = \sum_i c_i b_i$ 和 $y = \sum_j d_j b_j$ ，因此通过分配律，我们发现 $xy = \sum_{i,j} T_{i,j} c_i d_j$ 。将 $T_{i,j}$ 写为 $\sum_k t_{i,j,k} b_k$ ， xy 的范式由 $\sum_{i,j,k} c_i d_j t_{i,j,k} b_k$ 给出。基元素乘积的矩阵 T 是名称 `times_table_tac` 的来源。

例如，如果我们设置 $S = \mathbb{Z}[\sqrt{d}]$ ，我们可以选择 $\{1, \sqrt{d}\}$ 作为基向量，其乘法表（times table）如下

$$T = \begin{pmatrix} 1 & \sqrt{d} \\ \sqrt{d} & d \end{pmatrix}.$$

为了也支持在表达式中使用变量，如果 S 是度数为 d 的 R -模，我们可以为 $S[X_1, \dots, X_n]$ 构造一个范式，通过在多项式环 $R[X_{1,0}, \dots, X_{1,d}, X_{2,0}, \dots, X_{n,d}]$ 中工作并设置 $X_i = \sum_j X_{i,j} b_j$ 。对于多元多项式环，合适的范式是霍纳范式（Horner normal form）。

`times_table_tac` 的实现自下而上地为每个子表达式构造范式，在遇到和、标量倍数和范式的乘积时，将它们求值为新的范式。在求值期间进行标准化可确保项保持相对较小，避免了 `calc_tac` 在更复杂表达式上的问题。范式被记录为霍纳范式的向量，重用 `ring` 策略的实现来操作霍纳表达式 [15]。除了计算范式外，该策略还组装了一个证明，证明输入表达式等于其范式。构造证明项在 `Lean 3` 中往往比反射证明（proof by reflection）更快，因为元程序的解释器通常比内核归约（kernel reduction）更高效。基的数据及其相关的乘法表捆绑在一个结构 `times_table` 中：

```
structure times_table (ι R S : Type*) [fintype ι]
  [semiring R] [add_comm_monoid S] [module R S]
```

```
[has_mul S] :=
(basis : basis ι R S)
(table : ι → ι → ι → R)
(unfold_mul : ∀ x y k, repr basis (x * y) k =
  ∑ i j : ι, repr basis x i * repr basis y j *
    table i j k)
```

字段 `basis` 和 `table` 分别对应于上面的变量 b 和 t 。这里，`repr basis` 是一个线性等价，将向量 $x : S$ 发送到系数族 c 使得 $\sum i, c_i \cdot \text{basis } i = x$ 。

我们以扩展 `norm_num` 策略的形式为 `times_table_tac` 提供了一个用户界面。此策略旨在通过将目标或假设中出现的数字表达式重写为显式数字来评估它们。具体来说，`times_table_tac` 扩展评估形式为 `repr (times_table.basis T) x` 的表达式。因此，乘法表通过我们要标准化的表达式传递给策略。

请注意，`times_table` 的定义不假设 S 具有环结构。这允许我们使用该策略来证明环公理给定根据乘法表明定义的乘法，就像我们使用 `calc_tac` 在 `quad_ring` 上构造 `comm_ring` 实例一样。

由于 `quad_ring.calc_tac` 已被证明足以满足我们目前的目的，我们没有在实践中应用 `times_table_tac`，而是将其保留作为概念验证。从我们的测试用例中，我们确定随着表达式大小的增长，`calc_tac` 花费的时间明显多于 `times_table_tac`：在操作很少的目标上，例如 $Q(\sqrt{d})$ 中乘法的交换性或结合性，`calc_tac` 比 `times_table_tac` 快大约两倍，而在诸如 $x^4 - y^4 = (x - y)(x^3 + x^2y + xy^2 + y^3)$ 这样更大的目标上，该比率降至 $\frac{3}{2}$ 。尽管如此，这意味着两种策略都随着表达式的增长而出现明显的减速：它们需要超过一分钟的时间来解决这个相对简单的目标。不幸的是，减少像 $\sum_{i,j,k} c_i d_j t_{i,j,k} b_k$ 这样的求和的标准化步骤花费了足够多的时间，以至于 `calc_tac` 在我们开发中遇到的小型表达式上更快。我们认为，在实现 `times_table_tac` 的完整化身时，良好的缓存策略可以解决这些问题，因为在展开定义时，相同的表达式 c_i, d_j 和 $t_{i,j,k}$ 会倾向于重新出现。

7 Links to computer algebra systems

在上述大部分工作中，很多时间都花在了常规的、可以算法化完成的计算上。特别是，这些计算一旦执行，就可以比它们执行时更快地得到认证。例如，检查元素在理想中的成员资格，或更一般地检查由显式生成元列表给出的两个理想之间的包含关系或相等关系可能是一项非平凡的任务（取决于基础环），但只要环中可以进行有效的算术运算，它总是可以得到认证的。这就给形式化者提出了一个选择：要么在它们要被使用的证明助手中或在其元语言中实现这种算法的计算高效版本，要么在现有的计算机代数系统中外部进行这些计算，然后以某种方式导入并验证结果。

直接实现这些计算可能会极其困难且耗时。根据所需的计算，可能需要几个月的时间才能在证明助手中实现最先进的算法。从外部工具导入计算可以在一定程度上以手动方式完成，但如果涉及的术语很大，则会增加大量开销并降低可读性。

为了自动化在证明中途调用外部系统的过程，我们开发了一个策略，它使用 SageMath 计算机代数系统 [31] 作为后端，以确定更有用形式的数据的直接替换，要么是以某种方式简化的，要么是分解成使术语的某些属性更清晰的片段。主要策略 `replace_certified_sage_equality` 将在下面描述。它旨在成为一种灵活的方法，允许用户使用 SageMath 添加他们自己的策略来进行未经验证的计算，然后仅用几行代码就可以验证它们。该系统在目标上类似于 [18, 20]，但我们的目标是使我们的系统在某些方面具有更广泛的适用性。

该系统旨在具有灵活性，以便它可以用于实现许多符合此配置文件的策略。例如，我们可以实现策略来分解整数和多项式，分解为可除部分和余数，或者将矩阵替换为基变换矩阵乘以某种范式的矩阵。有关如何实现对 SageMath 的访问的详细信息可以在 section A 中找到。我们打算在我们工作中需要显式计算的部分使用这种范例，但尚未这样做。

Design. `replace_certified_sage_equality` 的签名如下：

```
meta def replace_certified_sage_equality
  (matcher : expr → tactic bool)
  (reify_type : Type*)
  (reify : expr → tactic reify_type)
  (sage_input : reify_type → tactic string)
  (output_type : Type*)
  [has_from_sage output_type]
  (convert_output : output_type → tactic expr)
  (validator : expr → expr → tactic (expr))
  (name : string)
  (wi : parse (tk "with" *> pexpr?) :
    tactic unit
```

这个 `replace_certified_sage_equality` 策略遍历目标，并查找 `matcher` 返回 `true` 的子表达式。然后使用 `reify` 将找到的第一个子表达式具体化为 `reify_type` 类型的显式数据；这意味着即使是不可计算的类型的术语，只要 `reify` 函数可以识别该术语，也可以被具体化。然后使用 `sage_input` 函数创建一个字符串，即传递给 SageMath 的命令。SageMath 的输出（通过其中一个后端）被读取为字符串，该字符串被解析为 `output_type`（此解析通过类型类处理，以便于解析诸如嵌套元组和列表之类的类型）。然后，`convert_output` 负责构造一个表示与最初匹配的输入相同类型的术语的表达式，它们之间的相等性随后由返回相等性证明的 `validator` 证明。然后将其重写以用 SageMath 返回的经过验证的版本替换最初匹配的表达式。最后两个参数用于解析策略之后可选的 `with "a string"`，这在自我替换版本中使用。

使用这个样板策略，使用 SageMath 分解自然数的策略只需大约 20 行即可实现（参见 section B），而实现者无需了解如何调用 SageMath 的详细信息。对于这个例子，我们将表示自然数的表达式具体化为自然数本身，并要求 SageMath 作为一个素因子和指数的列表对进行分解。然后我们将相应的因式分解项构造为幂的乘积。`mathlib` 策略 `norm_num` 用于证明原始表达式和结果表达式的等价性。

8 Discussion

8.1 Future work

除了第 4.4 节中提到的方法（尤其是使用闵可夫斯基界的方法）之外，还有几种很自然的方法可以扩展当前的工作。

可以进行修改以考虑其他莫德尔方程，包括以下内容。当 $d = -7$ 时，这将涉及考虑模 4 同余为 1 的判别式，因此在 $\text{quad_ring } \mathbb{Z} \ 1 \ ((d - 1) / 4)$ 上工作。这应该不会比上面的证明复杂多少。当 $d = -23$ 时，这将涉及处理 $h(K) = 3$ 的情况。为此，我们需要对类群的元素使用显式的理想代表元（原则上我们将计算这些代表元）。当 $d > 0$ 时，这将涉及更多工作，因为 $\mathbb{Q}(\sqrt{d})$ 的整数环的单位群（假设 d 不是完全平方数）此时是无限的。尽管模立方后单位的可能性仍然是有限的，但这在从主理想传递到元素时需要一些额外的案例分析，除了计算单位群的生成元的问题之外。

我们还可以考虑更一般形式的方程。上述讨论的技术仍然可以用于求解比莫德尔方程更一般的方程。例如， $y^2 = x^3 + d$ 中的指数 2, 3 可以更改，或者更一般地，可以考虑形式为 $y^d = f(x)$ 的超椭圆方程，其中 $f \in \mathbb{Q}[x]$ 是任意多项式。这涉及在代数 $\mathbb{Q}[x]/\langle f \rangle$ 中工作，并且至少需要在计算类群和单位群的验证算法方面取得重大进展。

在策略方面，很明显在使用显式数域和理想时出现的许多目标将具有相当特定的形式，例如检查数域中的某些等式或计算，检查两个理想的相等性，或找到它们的标准生成元集，检查类群中理想的等价性，或证明给定理想的非主理想性。用于解决此类目标的专用策略应该指日可待，并且在形式化该领域的论证时将极大地加快速度。

求解莫德尔方程的一个应用是能够确定所有具有指定坏还原（bad reduction）的 $\mathbb{Q}$ 上的椭圆曲线。这也可以在未来的工作中形式化。

8.2 Related work

以前关于丢番图方程的工作主要包括初等解析，例如费马、欧拉、高斯和勒让德的工作。有几种这类材料的形式化，涉及显式变量替换、检查素数模的解、二次互反律以及在 $\mathbb{Z}$ 内的下降。例如，费马大定理指数为 4 的情况已在几个证明助手中形式化，包括 Isabelle/HOL（见 [26]，指数为 3 的情况也一样）、Coq（见 [10]）和 Lean³。

从数理逻辑的角度来看，有几种涉及丢番图方程和问题的形式化（例如见 [17] 或 [27] 及其中的参考文献）。该领域与本文考虑的基本研究对象和问题有一些重叠，但在方法和目标方面仍然截然不同。

佩尔方程 $(x^2 - dy^2 = 1)$ 的解是 Freek Wiedijk 跟踪的形式化目标的“前 100 个”数学定理之一，并且该方程的某些方面已在 HOL Light、Isabelle/HOL、Metamath、Lean 和 Mizar 中形式化。这与计算实二次域中的单位群有关，单位群在求解 d 为正数的莫德尔方程时起作用。然而，Lean 的 `mathlib` 中的形式化仅专门针对 $d = a^2 - 1$

³https://github.com/leanprover-community/mathlib/blob/4e1eebe/src/number_theory/fermat4.lean

对于某些 $a \in \mathbb{Z}$ 的情况。这使得它只对求解莫德尔方程时出现的实二次域中极小的一部分有用，因此如果不进行推广，作为一种形式化，它在丢番图方程的应用中就不如原本可能的那样有用。

也有一些致力于形式化与费马大定理相关的倡议，最著名的是正在进行的“FLT regular”项目⁴，旨在形式化库默尔在 19 世纪关于常规素数指数的费马大定理的工作。

De Frutos-Fernández 形式化了数域的阿代尔 (adèles) 和伊代尔 (idèles) 理论 [7]，这提供了另一种更具理论性的思考理想类群的方法，并开始了类域论的发展。

8.3 Conclusion

我们已经成功地形式化了几个莫德尔方程的解，更重要的是，建立了可用于求解莫德尔方程的其他实例或一般丢番图方程的理论和策略。我们发现我们的大部分努力都投入到了计算的每一个细节上，例如理想的相等性，这通常留给计算机代数系统来完成，我们设计了一些策略的原型，以期在未来让这项工作变得更容易。另一方面，更具理论性的论证（例如由 $\sqrt{2}$ 范数的素性推导其本身的素性）则很顺利地形式化了。

总的来说，我们的开发占用了大约 6000 行代码，其中大约 1000 行涉及预备知识，其余大致平均分布在本文的第 3 到 6 节中。对于像这样的项目，作为一个整体，德布鲁因因子 (De Bruijn factor) 很难可靠地测量。这是因为这里形式化的结果在纸面上和形式系统中都有很多先决条件，没有一本书或一篇文章是从几乎没有数学背景开始并呈现这里形式化的结果的。因此，对德布鲁因因子的任何测量都严重依赖于作者选择将什么视为背景材料或非正式工作中的隐含内容。

Acknowledgments

Anne Baanen 由 NWO Vidi 拨款第 016.Vidi.189.037 号 (Lean Forward) 资助。Alex J. Best、Nirvana Coppola 和 Sander R. Dahmen 由 NWO Vidi 拨款第 639.032.613 号 (New Diophantine Directions) 资助。

我们要感谢 Jasmin Blanchette 对本文早期版本的评论。感谢匿名审稿人的宝贵意见。最后，我们要感谢 mathlib 社区在整个开发过程中的支持。

A Appendix: Access to systems

我们相信，使用免费和开源的计算机代数系统作为后端有助于确保许多用户能够潜在地使用这些策略。为了增加此类策略无需额外设置即可在特定系统上使用的可能性，我们实现了几个后端，用于在用户的机器上直接调用 SageMath，或通过 conda⁵ 安装。如果用户机器上没有安装 SageMath，我们添加了三个后端，通过使用 curl、ncat 或 openssl 中的一种，通过 <https://sagecell.sagemath.org> 访问互联网上公开可用的 SageMath 实例。后一种方法以前曾被由 Dhruv Bhatia 编写的 mathlib 策略 polyrith 使用。这些是我们选择将 SageMath 用于这些策略的主要

原因，尽管应该可以很容易地使用其他系统代替它而无需进行重大更改，因为接口是通过纯文本进行的。

除了确保可以使用各种访问 SageMath 的方式外，我们还以自我替换的方式设计了此策略。由于 SageMath 返回字符串输出，因此一旦策略运行一次，未来就可以直接为其提供适当的 SageMath 输出。

例如，使用通过此模式实现的自然数分解策略，建议用户使用包含 SageMath 显式输出的调用替换对策略的调用：

```
example : ¬ nat.prime 1111 := begin
  factor_nats, -- Try this: factor_nats with
    "[ (11, 1), (101, 1) ]"
  simp [nat.prime_mul_iff], -- a product can only
    be prime if all factors are one
end
```

B Appendix: Example of a CAS certification tactic for factoring

```
meta def tactic.interactive.factor_nats :=
  replace_certified_sage_equality
  (λ ex, do
    t ← infer_type ex,
    return $ t = `(nat))
N
(λ e, e.to_nat)
(λ n, return
  sformat!"print(list(ZZ({n}).factor()))")
(list (N × N))
(λ l, do
  ini ← l.mfoldl (λ ol ⟨p, n⟩, do
    P ← expr.of_nat `(N) p,
    N ← expr.of_nat `(N) n,
    return `((%ol : N) * (%P : N) ^ (%N :
  N))) `(1 : N),
  sl ← simp_lemmas.mk_default,
  prod.fst <$> simplify sl [] ini)
(λ o n, do
  (e1', p1) ← or_refl_conv norm_num.derive o,
  (e2', p2) ← or_refl_conv norm_num.derive n,
  is_def_eq e1' e2',
  mk_eq_symm p2 >=> mk_eq_trans p1)
"factor_nats"
```

References

- [1] Reynald Affeldt, Cyril Cohen, Marie Kerjean, Assia Mahboubi, Damien Rouhling, and Kazuhiko Sakaguchi. 2020. Competing Inheritance Paths in Dependent Type Theory: A Case Study in Functional Analysis. In *IJCAR 2020 (LNCS, Vol. 12167)*, Nicolas Peltier and Viorica Sofronie-Stokkermans (Eds.). Springer, 3–20. https://doi.org/10.1007/978-3-030-51054-1_1
- [2] Anne Baanen. 2022. Use and Abuse of Instance Parameters in the Lean Mathematical Library. In *ITP 2022 (LIPICs, Vol. 237)*, June Andronick and Leonardo de Moura (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 4:1–4:20. <https://doi.org/10.4230/LIPICs.ITP.2022.4>

⁴<https://github.com/leanprover-community/flt-regular>

⁵<https://docs.conda.io>

- [3] Anne Baanen, Sander R. Dahmen, Ashvni Narayanan, and Filippo A. E. Nuccio Mortarino Majno di Capriglio. 2021. A Formalization of Dedekind Domains and Class Groups of Global Fields. In *ITP 2021 (LIPIcs, Vol. 193)*, Liron Cohen and Cezary Kaliszyk (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, 5:1–5:19. <https://doi.org/10.4230/LIPIcs.ITP.2021.5>
- [4] Alan Baker. 1968. Contributions to the Theory of Diophantine Equations. II. the Diophantine Equation $y^2 = x^3 + k$. *Philosophical Transactions of the Royal Society of London. Series A, Mathematical and Physical Sciences* 263, 1139 (1968), 193–208. <http://www.jstor.org/stable/73429>
- [5] Michael A. Bennett and Amir Ghadermarzi. 2015. Mordell’s equation: a classical approach. *LMS Journal of Computation and Mathematics* 18, 1 (2015), 633–646. <https://doi.org/10.1112/S1461157015000182>
- [6] Henri Cohen. 2007. *Number theory: Volume I: Tools and Diophantine equations*. Graduate Texts in Mathematics, Vol. 239. Springer, Cham. <https://doi.org/10.1007/978-0-387-49923-9>
- [7] María Inés de Frutos-Fernández. 2022. Formalizing the Ring of Adèles of a Global Field. In *ITP 2022 (LIPIcs, Vol. 237)*, June Andronick and Leonardo de Moura (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 14:1–14:18. <https://doi.org/10.4230/LIPIcs.ITP.2022.14>
- [8] Leonardo de Moura, Soonho Kong, Jeremy Avigad, Floris van Doorn, and Jakob von Raumer. 2015. The Lean theorem prover (system description). In *Automated Deduction - CADE-25 (LNCS, Vol. 9195)*, A. P. Felty and A. Middeldorp (Eds.). Springer, 378–388. https://doi.org/10.1007/978-3-319-21401-6_26
- [9] Leonardo de Moura and Sebastian Ullrich. 2021. The Lean 4 Theorem Prover and Programming Language. In *Automated Deduction - CADE-28 (LNCS, Vol. 12699)*, André Platzer and Geoff Sutcliffe (Eds.). Springer, 625–635. https://doi.org/10.1007/978-3-030-79876-5_37
- [10] David Delahaye and Micaela Mayero. 2005. Diophantus’ 20th Problem and Fermat’s Last Theorem for $n = 4$: Formalization of Fermat’s Proofs in the Coq Proof Assistant. Draft. <http://arxiv.org/abs/cs/0510011>
- [11] Leonard E. Dickson. 1920. *History of the theory of numbers*. Vol. 2. Carnegie Institution of Washington.
- [12] David S. Dummit and Richard M. Foote. 2004. *Abstract algebra* (third ed.). John Wiley & Sons, Inc., Hoboken, NJ. xii+932 pages.
- [13] Manuel Eberl. 2017. Minkowski’s Theorem. *Archive of Formal Proofs* (July 2017). https://isa-afp.org/entries/Minkowskis_Theorem.html, Formal proof development.
- [14] Harold M. Edwards. 2000. *Fermat’s last theorem*. Graduate Texts in Mathematics, Vol. 50. Springer, Cham. xvi+410 pages. <https://doi.org/10.1007/978-0-387-95002-0> A genetic introduction to algebraic number theory, Corrected reprint of the 1977 original.
- [15] Benjamin Grégoire and Assia Mahboubi. 2005. Proving equalities in a commutative ring done right in Coq. In *TPHOLS 2005 (LNCS, Vol. 3603)*, J. Hurd and T. Melham (Eds.). Springer, Cham, 98–113. https://doi.org/10.1007/11541868_7
- [16] S. Lang. 2005. *Undergraduate Algebra*. Springer New York.
- [17] Dominique Larchey-Wendling and Yannick Forster. 2019. Hilbert’s Tenth Problem in Coq. In *FSCD 2019 (LIPIcs, Vol. 131)*, Herman Geuvers (Ed.). Schloss Dagstuhl–Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 27:1–27:20. <https://doi.org/10.4230/LIPIcs.FSCD.2019.27>
- [18] Robert Y. Lewis. 2017. An Extensible Ad Hoc Interface between Lean and Mathematica. *Electronic Proceedings in Theoretical Computer Science* 262 (Dec. 2017), 23–37. <https://doi.org/10.4204/EPTCS.262.4> arXiv:1712.09288 [cs].
- [19] Robert Y. Lewis and Paul-Nicolas Madelaine. 2020. Simplifying Casts and Coercions (Extended Abstract). In *Practical Aspects of Automated Reasoning (CEUR Workshop Proceedings, Vol. 2752)*, Pascal Fontaine, Konstantin Korovin, Ilias S. Kotsireas, Philipp Rümmer, and Sophie Tourret (Eds.). CEUR-WS.org, Aachen, Germany, 53–62. <http://ceur-ws.org/Vol-2752/paper4.pdf>
- [20] Robert Y. Lewis and Minchao Wu. 2022. A Bi-Directional Extensible Interface Between Lean and Mathematica. *Journal of Automated Reasoning* 66, 2 (May 2022), 215–238. <https://doi.org/10.1007/s10817-021-09611-1>
- [21] Louis Mordell. 1914. The Diophantine Equation $y^2 - k = x^3$. *Proceedings of the London Mathematical Society* s2-13, 1 (01 1914), 60–80. <https://doi.org/10.1112/plms/s2-13.1.60>
- [22] Louis Mordell. 1920. A Statement by Fermat. *Proceedings of the London Mathematical Society* s2-18 (1920), v–vi. <https://doi.org/10.1112/plms/s2-18.1.1-s>
- [23] Louis Mordell. 1923. On the Integer Solutions of the Equation $ey^2 = ax^3 + bx^2 + cx + d$. *Proceedings of the London Mathematical Society* s2-21, 1 (01 1923), 415–419. <https://doi.org/10.1112/plms/s2-21.1.415>
- [24] Louis Mordell. 1969. *Diophantine equations*. Academic Press.
- [25] J. Neukirch. 1999. *Algebraic number theory*. Fundamental Principles of Mathematical Sciences, Vol. 322. Springer, Cham. xviii+571 pages. <https://doi.org/10.1007/978-3-662-03983-0> Translated from the 1992 German original and with a note by Norbert Schappacher, With a foreword by G. Harder.
- [26] R. Oosterhuis. 2007. Mechanised theorem proving: exponents 3 and 4 of Fermat’s last theorem using Isabelle. MSc thesis. Faculty of Science and Engineering, Groningen. <https://fse.studenttheses.ub.rug.nl/8392/>
- [27] Karol Pāk and Cezary Kaliszyk. 2022. Formalizing a Diophantine Representation of the Set of Prime Numbers. In *ITP 2022 (LIPIcs, Vol. 237)*, June Andronick and Leonardo de Moura (Eds.). Schloss Dagstuhl – Leibniz-Zentrum für Informatik, Dagstuhl, Germany, 26:1–26:8. <https://doi.org/10.4230/LIPIcs.ITP.2022.26>
- [28] Daniel Selsam, Sebastian Ullrich, and Leonardo de Moura. 2020. Tabled Typeclass Resolution. *CoRR* abs/2001.04301 (2020). arXiv:2001.04301 <https://arxiv.org/abs/2001.04301>
- [29] Harold Stark. 1973. Effective estimates of solutions of some Diophantine equations. *Acta Arithmetica* 24, 3 (1973), 251–259. <http://eudml.org/doc/205226>
- [30] The mathlib Community. 2020. The Lean Mathematical Library. In *CPP 2020 (Certified Programs and Proofs)*, J. Blanchette and C. Hrițcu (Eds.). ACM, 367–381. <https://doi.org/10.1145/3372885.3373824>
- [31] The Sage Developers. 2022. *SageMath, the Sage Mathematics Software System (Version 9.7)*. <https://www.sagemath.org>